

SDP



Loop Engineering

Designing Agent Systems That Run Reliably

Autonomous by design. Verifiable by default.

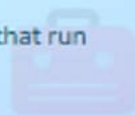
"What makes an agent loopy — and how do we engineer it to run reliably?"

Single-shot agents wait for the next prompt.

Loopy agents verify their own progress and keep going until the goal is met.

Platform Engineer

Designing shared agent pipelines that run unattended across teams



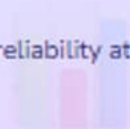
Senior Developer

Moving beyond one-shot Copilot into verified autonomous workflows



Team Lead

Governing agent cost, safety, and reliability at scale



8x code output

Anthropic teams with loop-managed autonomy — Claude writes >80% of merged production code

18 min advisory to PR

Security fix fully automated: scan → fix → critic → verified PR, zero human prompts

Most agentic failures

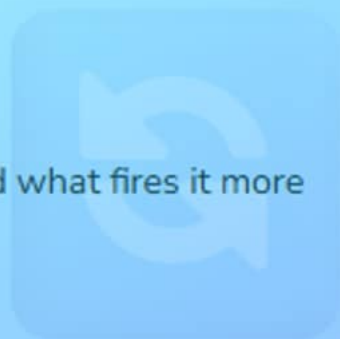
Trace to missing circuit breakers and watchdogs — the controls teams skip first

PART 1

The Loop

Definition + Trigger

What is a loopy agent and what fires it more than once



PART 2

Multi-Agent Design

Handoff + Topology

How agents coordinate state, escalate, and organize

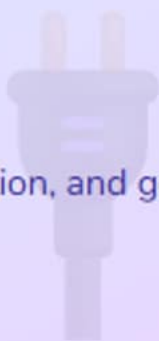


PART 3

Production Controls

Don't skip these

Circuit breaker, bounded execution, and goal-watchdog

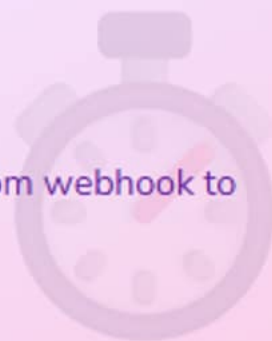


PART 4

Wiring It Together

18 min, zero prompts

Full multi-agent pipeline from webhook to verified PR



Click any section to jump directly there

The Loop

What IS a loopy agent — and what triggers it to run more than once?



Definition

Act → verify → repeat until goal met — then stop



Trigger

Copilot Automations — native, no infrastructure



Simple Path

Three agent files + shared state, no framework

loop-engineering

↳ act → verify → repeat until goal met

What Is a Loopy Agent?

One sentence. Everything else in this talk follows from it.

Loop

An agent that continues to act and verify in a cycle until the goal is demonstrably met — then stops.

⚡ Triggered once

A Copilot Automation, webhook, or schedule fires the loop — no human kick required

🔄 Runs until verified

The agent reasons, acts, and checks its own goal condition — not just assumes it's done

● Stops on proof

The loop terminates when the goal is demonstrably true, or when controls say stop

vs Not orchestration

Orchestration runs once. A loopy agent runs until the work is verifiably complete.

💡 The hardest engineering challenge is not starting the loop — it is stopping it correctly.

Copilot Automations — The Native Loop Trigger

Repository → Agents tab → Automations → Create new

name: Daily Issue Triage

trigger:

type: schedule
interval: daily
time: "00:00 UTC"

prompt: |

Review all open issues.
Apply 'stale' to any with no activity
for 3 days. Close issues stale 7+ days
with a polite note.

tools:

- label_issues
- close_issues
- post_comments



Schedule triggers

Every hour · Every day · Every week — no cron syntax, no external scheduler



Event triggers

Issue created · PR opened · PR synchronized — fires on real repository activity



What makes it loopy

Without this trigger, the agent runs once and stops. This is what re-fires the loop.

The Orchestrator — Loop Logic in Plain Instructions

.github/agents/loop-orchestrator.agent.md

```
---  
description: Loop orchestrator  
tools: [read_file, write_file, github]  
---  
1. Read `.github/loop-state.json`.  
2. If iteration_count ≥ max_iterations,  
   write an escalation note and stop.  
3. Check all goal.conditions – if all  
   are true, write a summary and stop.  
4. For each task in state.pending,  
   invoke the right worker agent.  
5. Update state.results and increment  
   iteration_count.  
6. Repeat from step 2.
```

Trigger (step 1)

Reads shared state — the Automation already fired this agent on schedule

Goal + Verify (steps 2–3)

Checks verifiable conditions before delegating — stops on proof, not assumption

Execute + Control (steps 4–6)

Delegates to workers, updates state, and caps itself at max_iterations

The Worker — Accepts a Task, Returns a Result

.github/agents/loop-reviewer.agent.md

```
---  
description: Code review worker  
tools: [read_file, write_file,  
        search_code, github]  
---
```

1. Read your task from
 .github/loop-state.json
 (tasks.pending[0]).
2. Perform the code review
 for the assigned PR.
3. Write findings back to
 state.results.review.
4. Set tasks.complete[] and
 remove from tasks.pending.
5. Update goal.conditions
 [review_approved] to true
 if review passes.



Reads task from shared state

The state file is the handoff — no direct agent-to-agent call needed



Writes result back to state

Orchestrator reads results on next iteration and decides what to do next



Updates goal conditions

Workers own their piece of the goal — the orchestrator just checks the total

loop-state.json — The Shared Brain

.github/loop-state.json

```
{
  "iteration_count": 2,
  "max_iterations": 5,
  "goal": {
    "conditions": [
      { "name": "review_approved",
        "met": true },
      { "name": "tests_passing",
        "met": false }
    ]
  },
  "tasks": {
    "pending": ["test_run"],
    "complete": ["code_review"]
  },
  "results": {
    "review": "No critical issues"
  }
}
```

All agents read from here

The orchestrator, reviewer, and tester all read the same file — no direct coupling

All agents write back

Any writable surface works: JSON file, GitHub Issue, database row, or API

State is the audit trail

iteration_count, results, and goal.conditions give full provenance without extra tooling

PART 2

Multi-Agent Design

How agents hand off work, track state, and know when to escalate



Structured Handoff

State entries with intent, constraints,
escalate_on



Lifecycle States

Idle → Suspended: where loops silently
fail



Topology

Flat-pool, hierarchical, or pipeline —
your choice

`multi-agent-design`

`↳ handoff → escalate → verify → next agent`

Two Ways to Pass Work Between Agents

Simple Path

Write to loop-state.json

tasks.pending, goal.conditions, results

Worker reads its task

No direct call — state is the handoff

Key fields to include

intent · constraints · escalate_on

Structured Path

Typed task envelope (JSON)

task_id · trace_id · sender · receiver

escalate_on conditions

explicit triggers that return control upward

on_complete routing

declarative: who gets the baton next

```
{ "intent": "review_pr",  
  "escalate_on": ["deadline_exceeded"],  
  "on_complete": { "trigger": "tester" } }
```

Agent Lifecycle — The State Machine Every Loop Needs

Most silent failures live in the Idle → Suspended gap

✗ Healthy Execution

- 1 **Created**
Task received from orchestrator
- 2 **Initialized**
Context loaded, tools verified available
- 3 **Active → Terminated**
Goal verified; audit trail written
- 4 **Result in state store**
Orchestrator proceeds to next task

✓ Loop continues or terminates cleanly

✓ Failure Path

- 1 **Created**
Task received from orchestrator
- 2 **Active**
Agent begins ReAct cycle
- 3 **Idle**
Waiting for tool response — watchdog timer starts here
- 4 **Suspended**
Circuit breaker fires — without this, Idle runs until manual stop

⚡ Circuit breaker required — covered in Section 3

Idle → Suspended: where most production loop failures begin

Three Topologies — Pick One Before Writing Any Agent



Flat Pool

One orchestrator dispatches to N identical workers; results aggregated. Use for parallelizable homogeneous tasks.

`.github/agents/` = orchestrator + N workers

Aggregate with majority-vote or first-success



Hierarchical

Orchestrator → specialized workers → critic. Use when tasks need specialization and a quality gate.

Workers have distinct capabilities

Critic validates before accepting output

Recommended default for engineering pipelines



Pipeline

Each stage strictly depends on previous output. Use when the workflow is sequential by nature.

analyze → implement → test → review

Stage failure triggers rollback or escalate

Four Fields That Make Handoff Traceable



trace_id

Links every agent action in this session — makes post-hoc debugging possible without extra tooling



intent

What the receiving agent is being asked to do — not how, just what. Keeps agents swappable.



escalate_on

Explicit conditions that return control to orchestrator or human — never implicit, never ad hoc



on_complete

Who receives the baton when this agent finishes — makes the pipeline declarative rather than imperative



These four fields work whether the handoff is a JSON file, a state store entry, or a structured task envelope.

Production Controls

The three controls most teams skip and most failures trace back to



Circuit Breaker

Hard stop on iterations, time, and token cost



Bounded Execution

Budget caps per agent in every task delegation



Goal-Watchdog

Stall detection when Idle → Suspended silently fails

`production-controls`

↳ most failures trace here – catch them before deploy

The Three Controls That Separate Demos from Deployments



Circuit Breaker

Hard-stops the loop when safety or cost thresholds are hit — regardless of what the agent believes about its own progress.

max_iterations: stop after N attempts

timeout_minutes: hard wall-clock limit

max_cost_usd: pause for human review



Bounded Execution

Every agent receives an explicit token budget and deadline in its task. Agents that exceed their budget must escalate, not continue.

Per-agent token cap in state entry

Deadline_utc in task envelope

Escalate on budget exceeded



Goal-Watchdog

Monitors the loop at a fixed interval. If no measurable progress has been made after N iterations, escalates rather than running forever.

Checks goal.conditions each iteration

Detects stall in Idle → Suspended gap

Escalates to human on stall threshold

Loop Readiness Score — Run Before Deploying Any Loop

```
npx @cobusgreyling/loop-audit . --suggest
```

Loop Audit Results

[Exit Conditions]

- ✓ goal.verifiable_conditions defined
- ✗ controls.circuit_breaker missing
- ✗ controls.bounded_execution missing
- ⚠ controls.goal_watchdog missing

[Agent Communication]

- ✓ escalate_to defined on agents
- ⚠ trace propagation not configured

[Cost & Safety]

- ✗ max_cost_usd not set
- ✓ timeout_minutes < 120

✗ 3 failures — do not deploy

✗ FAIL: missing circuit_breaker

No hard stop — one stalled tool call becomes an unbounded cost event

✗ FAIL: max_cost_usd not set

Uncapped budget is a production incident waiting to happen

⚠ WARN: watchdog not configured

Stall detection missing — Idle → Suspended failures will run silently until the circuit breaker fires

Wiring It Together

18 minutes. Zero human prompts. From webhook trigger to verified PR.



The Outcome

Runtime table: T+0 to T+18, row by row



Agent Files

Who participates and how they share state



Audit Trail

Full provenance via trace_id, trigger to termination

wiring-it-together

↳ 18 minutes · zero prompts · verified

18 Minutes. Zero Human Prompts. Verified.

Security advisory published → CVE resolved → PR passing CI

```
$ GitHub: security_advisory.published [CRITICAL] – CVE in dependency
```

```
🤖 Copilot Automation fires loop-orchestrator
```

```
✓ T+1 min: Orchestrator dispatches to vuln-scan-agent (8k token budget)
```

```
✓ T+3 min: Scan agent returns affected_files_report
```

```
✓ T+7 min: Fix-proposal agent returns proposed_diff
```

```
✓ T+10 min: Test-runner validates fix against affected test suite
```

```
✓ T+11 min: Security-critic approves (confidence 0.91 > 0.85 threshold)
```

```
✓ T+12 min: PR-author opens PR with label [security, automated]
```

```
✓ T+18 min: CI passes. All 4 goal conditions verified true. Loop terminates.
```

```
Audit trail written: trace_id · per-agent token spend · decision chain
```

0 human prompts · \$3.80 total cost · full provenance via trace_id

Five Agent Files. One Shared State. No Framework.

.github/agents/

loop-orchestrator.agent.md

Reads state, dispatches to workers, checks all goal conditions

vuln-scan-agent.agent.md

Identifies affected files — writes affected_files_report to state

fix-proposal-agent.agent.md

Generates dependency fix — writes proposed_diff to state

security-critic-agent.agent.md

Validates fix quality and test coverage — approves or rejects

pr-author-agent.agent.md

What Makes It Work

Copilot Automation

Fires loop-orchestrator on security_advisory.published event

loop-state.json

All five agents read and write to the same file — no direct coupling

Goal conditions

pr_opened · ci_passing · original_cve_resolved ·
no_new_cves

Circuit breaker

max_iterations: 8 · timeout: 45 min · max_cost_usd: \$4.00

Scale up: same pattern,

From One-Shot Prompts to Loopy Agents

Before

Hand-prompt every step; agent stops when it thinks it's done

No recurring trigger — workflows run once and wait for a human to restart

Ad hoc messages between agents — no trace ID, no escalation path

No iteration cap — a stalled tool call runs until manual cancellation

After

Trigger once; agent loops until goal condition is verifiably true

Copilot Automation fires on schedule or on repository events — no human restart

Typed state entries with trace ID, constraints, and escalate_on fields

Circuit breaker, bounded execution, and goal-watchdog enforce hard limits

8x

code output — Anthropic teams with loop-managed autonomy

18 min

advisory to verified PR — zero human prompts

0

human prompts from trigger to verified completion

✓ What You Can Do Today

Today

- ❑ Create a Copilot Automation (Agents tab → Create new → daily schedule + triage task) — minimum viable loop trigger, no code required
- ❑ Write one recurring task as a verifiable goal condition — not a description, a checkable criterion

This Week

- ❑ Wire a three-agent pipeline (orchestrator + worker + critic) with a shared loop-state.json
- ❑ Run `npx @cobusgreyling/loop-audit` . --suggest for a Loop Readiness Score on an existing workflow

This Month

- ❑ Add circuit breaker, bounded execution, and goal-watchdog to every unattended agent loop
- ❑ Integrate loop-audit into CI as a pre-merge gate — fail the build on any FAIL-severity check

🔑 Key Takeaway

Every loopy agent starts with a Copilot Automation, a verifiable goal, and a stop condition.

Official Documentation

[Creating automations with Copilot cloud agent](#)

Scheduled and event-triggered automations: hourly, daily, weekly, issue/PR events

[Schedule and automate tasks with Copilot cloud agent](#)

June 2026 launch of Copilot Automations for private and internal repositories

Open Source & Guides

[cobusgreyling/loop-engineering](#)

Practical patterns, starters & CLI tools: loop-audit, loop-init, loop-cost

[gim-home/herald](#)

Agent management harness: lifecycle state machine, coordination primitives

[The Art of Loop Engineering — LangChain Blog](#)

Custom termination logic, dynamic replanning, nested loops, and state management

[Agentic Loops: From ReAct to Loop Engineering \(2026 Guide\)](#)

Loop anatomy, pattern taxonomy, and why loop engineering supersedes prompt engineering



Loop Engineering

Autonomous by design. Verifiable by default.

The Definition

A loopy agent continues to act and verify until the goal is demonstrably met — then stops.

The Trigger

Copilot Automations: Agents tab → Create new → pick a schedule. No infrastructure required.

The Proof

18 minutes, zero human prompts: from security advisory to verified PR, fully automated.

What recurring task on your team could run as a loopy agent starting tomorrow?